GUIDE 5 OF 8 · AI LEARNING SERIES

CrewAI

Build teams of AI agents that collaborate to complete complex tasks

01 What is CrewAI?

CrewAI is an open-source Python framework for orchestrating multiple AI agents that work together as a **"crew"**. Each agent has a role, a goal, and a backstory. Agents are assigned tasks and can use tools. A Process (sequential or hierarchical) determines how agents collaborate.

> **REAL-WORLD ANALOGY**
>
> Think of it like hiring a team: a Research Analyst gathers info, a Data Scientist analyzes it, a Writer drafts the report, and a Manager reviews. Each has a specialty. CrewAI automates this delegation.

02 Core Concepts

Concept	What It Is	Example
Agent	An LLM with a role, goal, backstory, and tools	Research Analyst, Code Reviewer, Data Scientist
Task	A specific job assigned to an agent with expected output	"Analyze Q3 sales data and summarize top trends"
Tool	A function the agent can call to interact with the world	Web search, file read, API call, code execution
Crew	The team of agents + their tasks + a process	Your complete multi-agent workflow
Process	How tasks are executed	Sequential (one after another) or Hierarchical (manager delegates)

03 Installation and Quick Start

```
# Installation
pip install crewai crewai-tools
```

```
# Complete CrewAI example — Research + Write
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool

# Define Tools
search_tool = SerperDevTool()  # Web search
```

```
# Define Agents
researcher = Agent(
    role="Senior Research Analyst",
    goal="Uncover cutting-edge developments in AI",
    backstory="Expert at finding and synthesizing information",
    tools=[search_tool],
    verbose=True,
    llm="gpt-4o-mini"
)

writer = Agent(
    role="Tech Content Writer",
    goal="Create engaging content from research findings",
    backstory="Expert writer who makes complex topics accessible",
    verbose=True
)

# Define Tasks
research_task = Task(
    description="Research the latest developments in {topic}.",
    expected_output="Bullet-point summary of top 5 developments",
    agent=researcher
)

write_task = Task(
    description="Write a 500-word blog post based on research findings.",
    expected_output="A polished blog post with intro, body, conclusion",
    agent=writer
)

# Create and Run the Crew
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,  # researcher first, then writer
    verbose=True
)

result = crew.kickoff(inputs={"topic": "Agentic AI in 2025"})
print(result.raw)
```

04 Process Types

Sequential Process

Tasks run one after another in order. Output of each task is available to the next agent as context. Simplest and most common pattern.

Hierarchical Process

A manager agent (powered by an LLM) decides which agent gets which task, reviews outputs, and coordinates work — like a real project manager. Requires manager_llm.

```
# Hierarchical crew
crew = Crew(
    agents=[researcher, analyst, writer],
    tasks=[research_task, analysis_task, write_task],
    process=Process.hierarchical,
    manager_llm="gpt-4o",  # smart manager model
    verbose=True
)
```

05 Built-in CrewAI Tools

Tool	What It Does
SerperDevTool	Google search via Serper API
WebsiteSearchTool	Semantic search within a specific website
FileReadTool	Read file contents
DirectoryReadTool	List and read files in a directory
CodeInterpreterTool	Execute Python code
PDFSearchTool	Semantic search within PDFs
CSVSearchTool	Semantic search within CSV files
YoutubeVideoSearchTool	Search within YouTube video transcripts
GithubSearchTool	Search GitHub repositories
ScrapeWebsiteTool	Scrape and extract text from a URL
Custom Tool (BaseTool)	Create your own tool by subclassing BaseTool

06 Advanced Features

Memory

CrewAI agents can have memory that persists between tasks or crew runs. Enable with memory=True on the Crew. Uses embeddings to store and retrieve past interactions.

Flows — Event-Driven Orchestration

CrewAI Flows (newer feature) allow you to build event-driven pipelines where crews and individual steps are connected with conditional logic using @start(), @listen(), @router() decorators.

```
# CrewAI Flow example
from crewai.flow.flow import Flow, start, listen, router
```

```
class ContentFlow(Flow):
    @start()
    def research(self):
        return research_crew.kickoff()

    @listen(research)
    @router()
    def check_quality(self, research_result):
        if len(research_result.raw) > 500:
            return "write"
        return "retry"

    @listen("write")
    def write_content(self, research_result):
        return writing_crew.kickoff(inputs={"research": research_result})
```

When to Use CrewAI

Use CrewAI When	Use Single Agent When
Task requires multiple distinct specializations	Single clear task or question
Work can be parallelized or pipelined	Speed matters most (agents add latency)
Quality benefits from review/critique cycles	Simple tool use or retrieval
Complex workflows with conditional routing	Budget is tight (multiple LLM calls)